

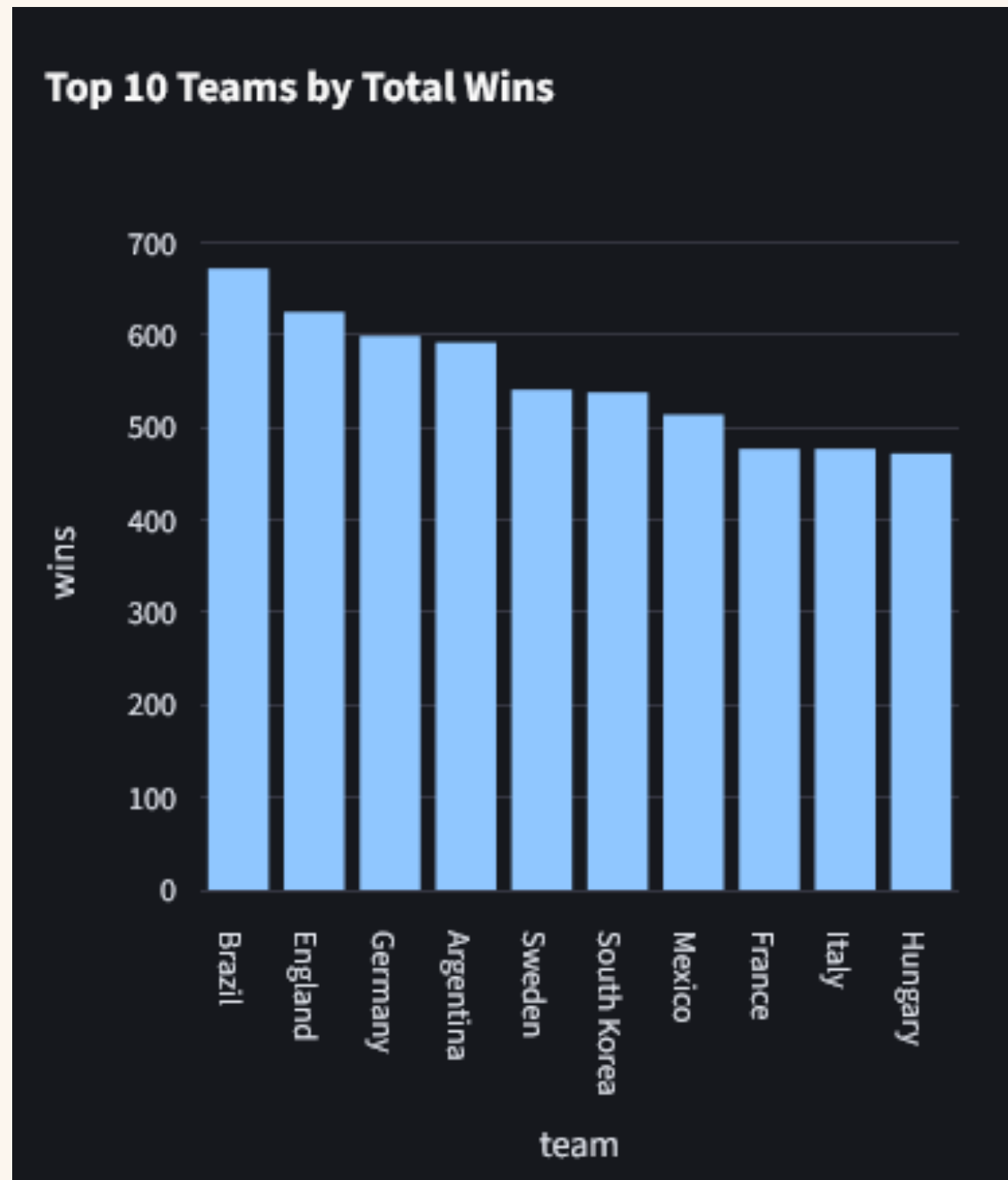
LLM-støttede dashboard

Andreas B. Berg

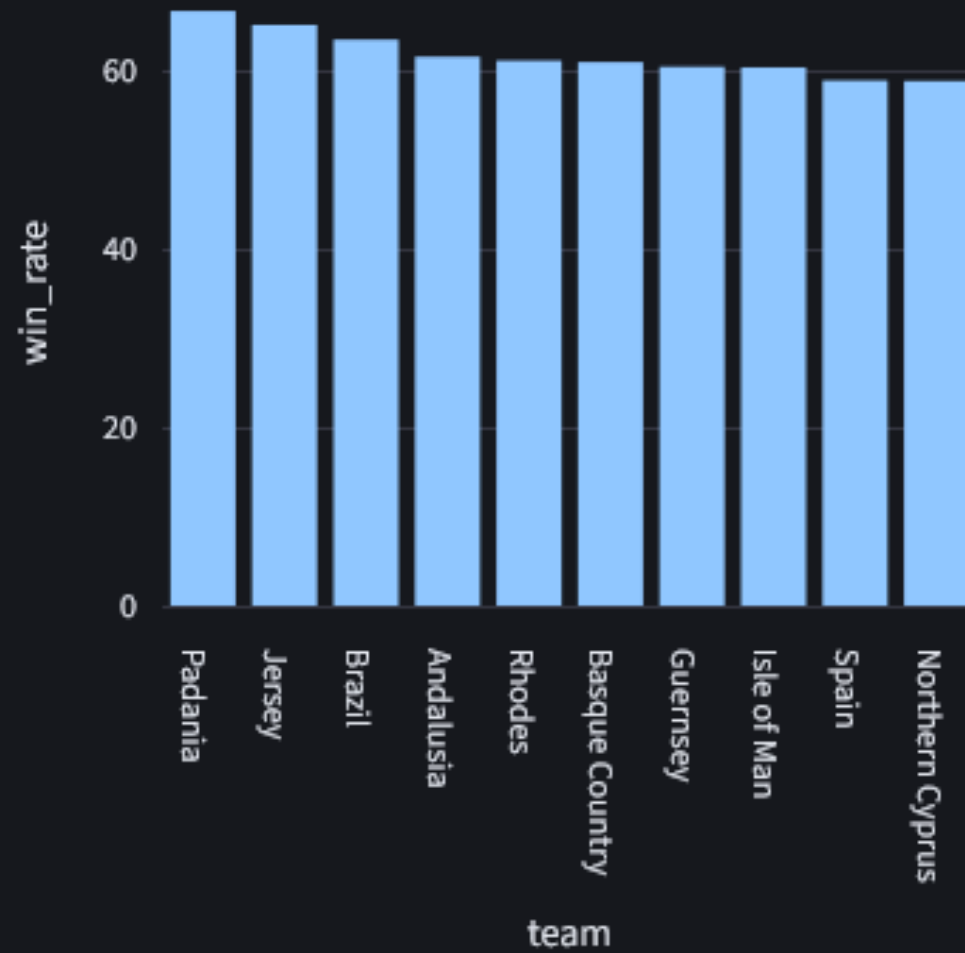
Agenda

1. Hvorfor
2. Demo
3. Hvordan
4. Hva må man tenke på?

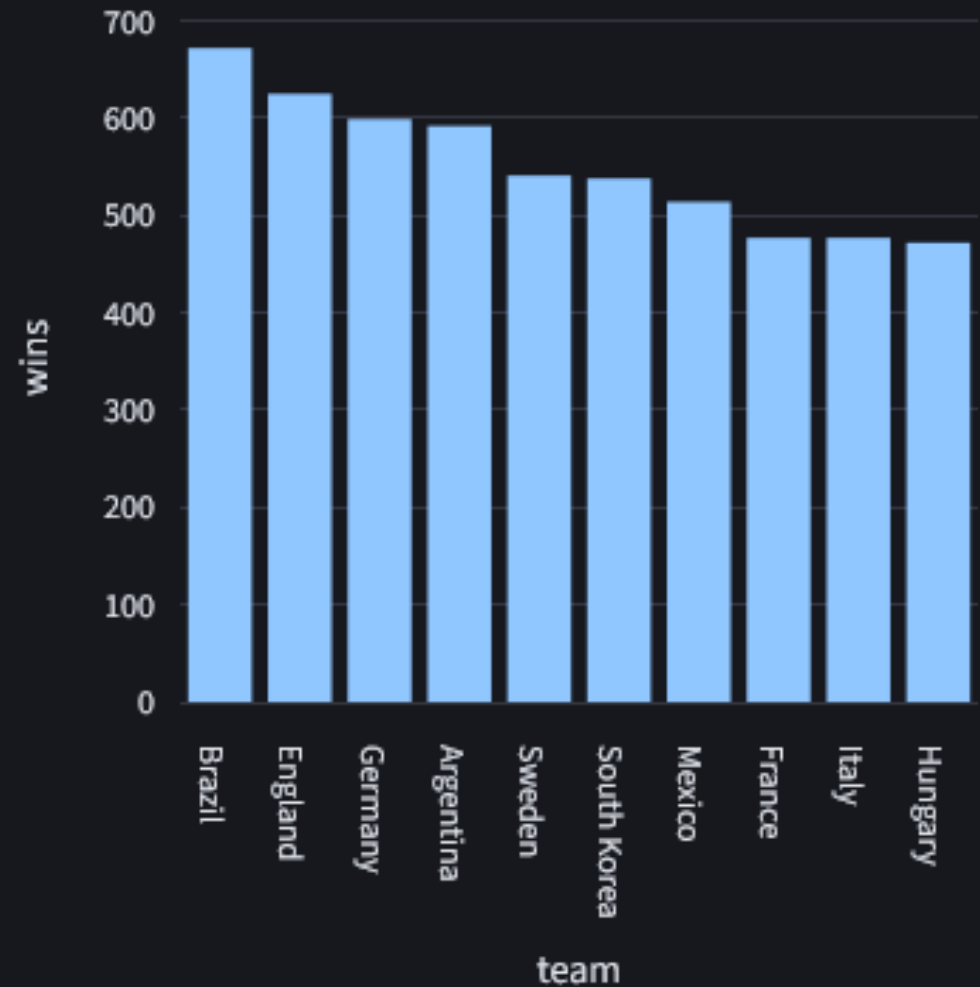
Hvorfor?



Top 10 Teams by Win Rate



Top 10 Teams by Total Wins



WC-preparation Dashboard

Teams

Choose opti... ▾

League

A ▾

Season

2022/2023 ▾

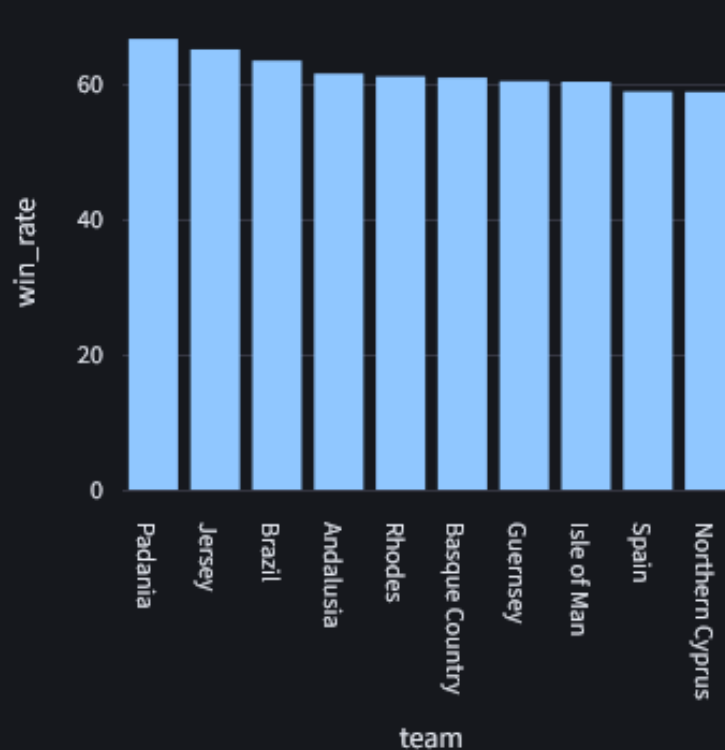
Matchday

1 ▾

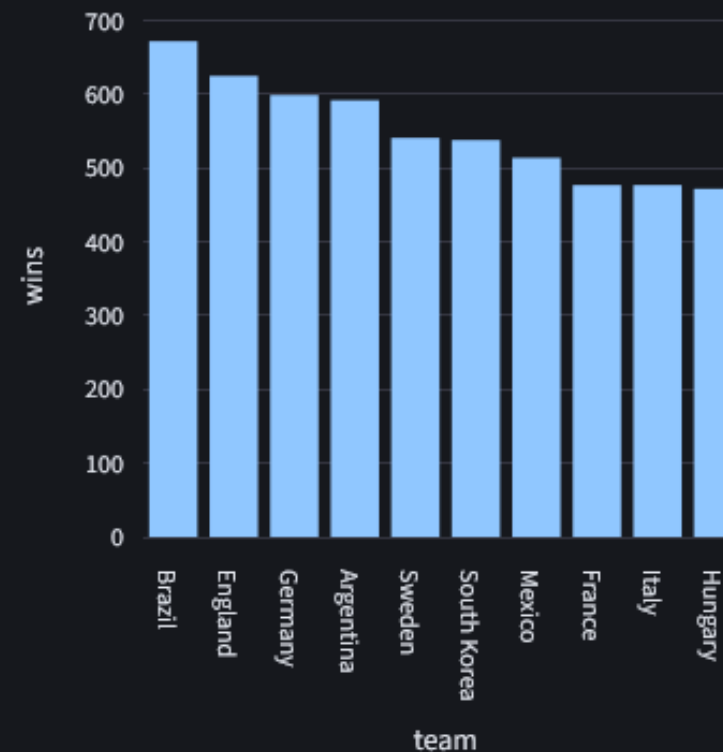
Top N teams

5

Top 10 Teams by Win Rate



Top 10 Teams by Total Wins



Demo

Kode tilgjengelig på:

[https://github.com/BollaBerg/LLM-supported-dashboards---](https://github.com/BollaBerg/LLM-supported-dashboards---Systek-fagdag-2026)
Systek-fagdag-2026

Hvordan?

Agenten

```
1 from google.adk.agents.llm_agent import Agent
2
3 from football_agent.tools import query_database
4
5 root_agent = Agent(
6     model="gemini-2.5-flash",
7     name="football_agent",
8     description="An assistant that answers questions about football results.",
9     instruction="""
10 Answer user questions using the tools you have available. If you don't know
11 the answer, clearly state that you don't know. Always use the tools when they
12 are relevant to the question.
13
14 You have access to run queries against a database. Make sure the query is
15 valid SQL. NEVER insert any data into the database, only SELECT from it.
16 """,
17     tools=[query_database],
18 )
```

Tools

```
1 def query_database(query: str) -> dict:
2     """
3     Query the database with the given query string and return the results as a Mar
4
5     The database is an SQLite3 database with information about Football results. T
6     - Table: results
7       - Columns: id, date, home_team, away_team, home_score, away_score
8
9     Example query: "SELECT home_team, away_team, home_score, away_score FROM resul
10
11     Args:
12         query (str): The SQL query to execute against the database.
13
14     Returns:
15         dict: status and result or error message.
16     """
17     database_path = Path(__file__).parent.parent.parent / "data" / "database.db"
18     try:
```

Runner

```
1  async def run_async(session_id: str, new_message: str):
2      content = types.Content(
3          role="user",
4          parts=[types.Part.from_text(text=new_message)]
5      )
6
7      raw_stream = _runner.run_async(
8          user_id="user_123",
9          session_id=session_id,
10         new_message=content,
11     )
12
13     async for message in raw_stream:
14         if message.content
15         and message.content.parts
16         and message.content.parts[0].text:
17             yield message.content.parts[0].text
```

Hva må man tenke på?

Pris

- Tokens koster
- Prompt og verktøybeskrivelser blir også tokens
- Mitigering:
 - Optimaliser prompt
 - Optimaliser verktøybeskrivelser
 - Cache svar der det er mulig
 - Lær fra brukerne

Hallusinering

- LLM-er kan finne på informasjon
- Mitigering:
 - Kort avstand fra data til svar
 - Dobbeltsjekk der mulig

Alt er async

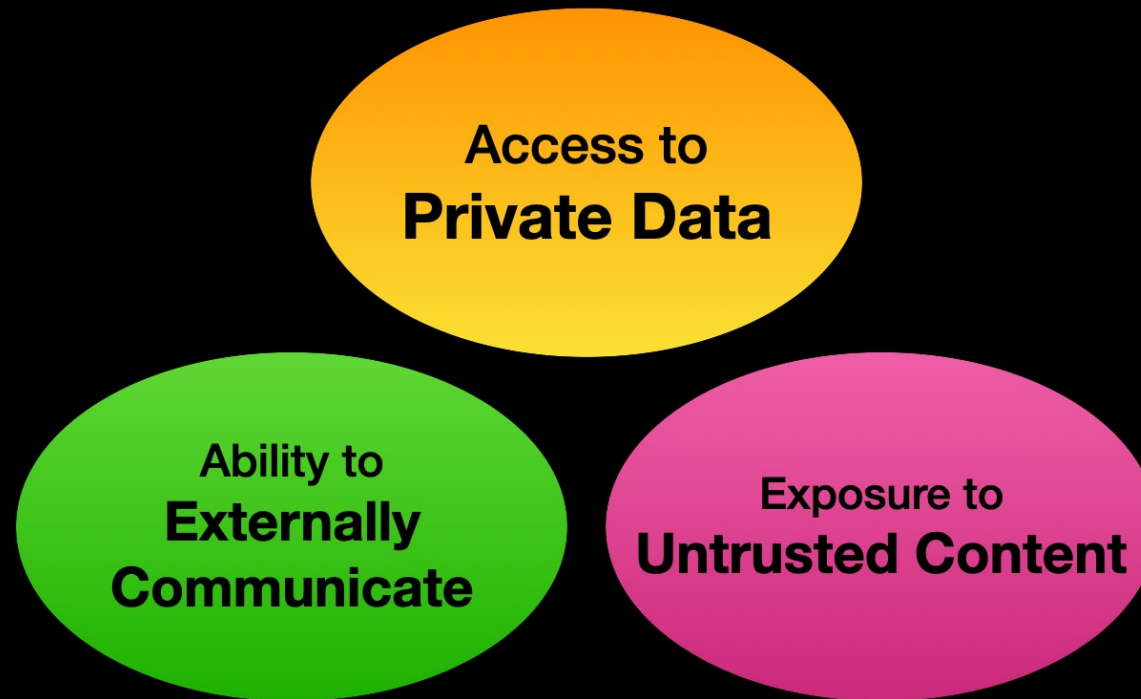
- Krever litt mer å sy sammen tools og AI-svar
- Vanskelig å postprosessere svar
- Mitigering:
 - Returner strings der mulig
 - Bruk verktøy som ikke krever mye etterbehandling
 - Legg inn forsinkelser der det er behov for det

Sikkerhet

- Agenter gjør som de får beskjed om
- Instruksjoner *kan* og *vil* overstyres
- Verktøy kan misbrukes

Sikkerhet

The lethal trifecta



Sikkerhet

- Mitigering:
 - Ikke stol på prompts
 - Ikke gi LLM flere tilganger enn brukeren
 - Flytt sikkerheten ut av LLM-nivå

Takk for meg